

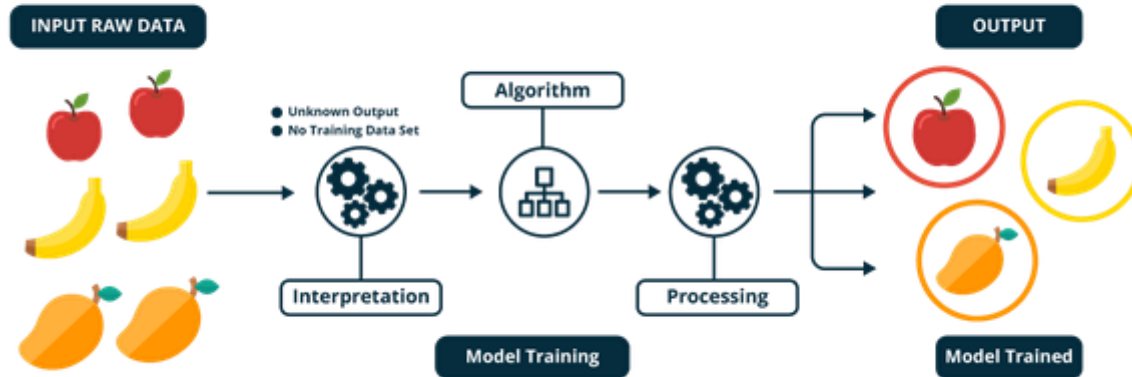
Agenda

- Unsupervised Learning
- Clustering in MachineLearning
- K-Means Clustering
- Hierarchal Clustering

1. Unsupervised Learning

Unsupervised Learning:

- **Unsupervised Learning** is a type of machine learning in which the algorithm is not provided with any pre-assigned labels or scores for the training data.
- As a result, **Unsupervised Learning** algorithms must first self-discover any naturally occurring patterns in that training data set.



UNSUPERVISED LEARNING

Unsupervised learning refers to the use of artificial intelligence (AI) algorithms on non-labelled dataset where the task is done by identifying the patterns in the dataset. The algorithms are concerned in classifying, labelling and grouping data without any external guidance in performing the task. The patterns of the data can be deduced by recognizing the relation between each sample in the dataset and the remaining samples in the feature space.

Unsupervised learning is way more complex than supervised learning in terms of computation and processing.

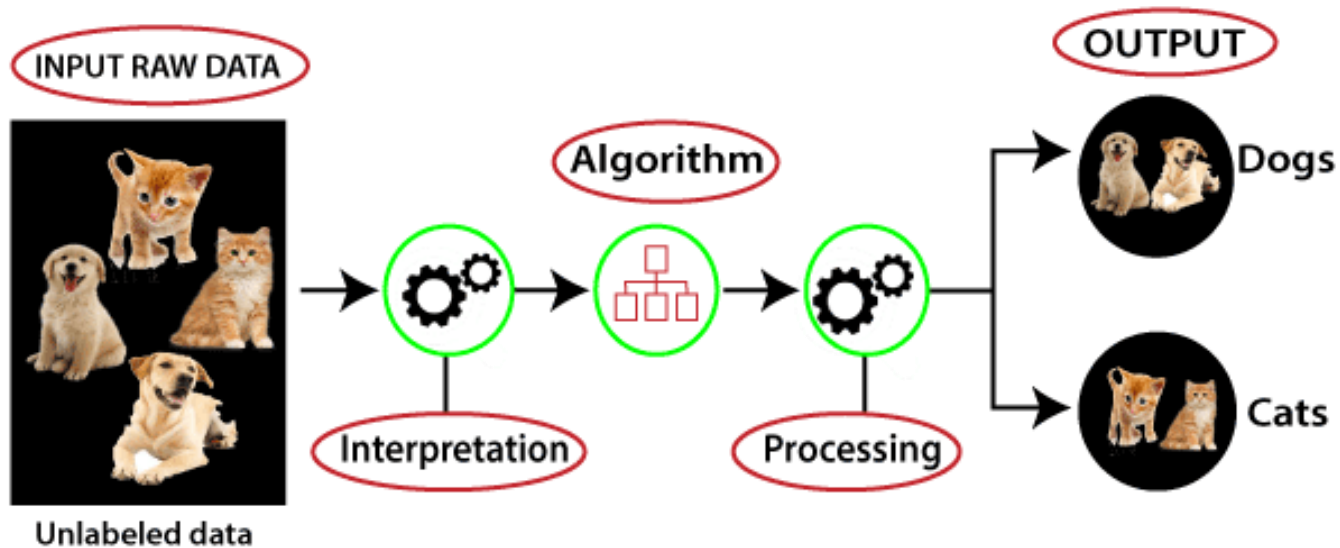
Unsupervised Algorithms:

- 1- Clustering (K-means, Hierarchical, DBSCAN, GMMs)
- 2- Association Rules
- 3- Dimensionality Reduction (PCA)

Example:

- Suppose that the Unsupervised Learning algorithm is given an input dataset containing images of different types of cats and dogs.
- The algorithm is never trained upon the given dataset, which means it does not have any idea about the features of the dataset.
- The task of the Unsupervised Learning algorithm is to identify the image features on their own.
- It will perform this task by clustering the image dataset into the groups according to similarities between images.

Example:



Example:

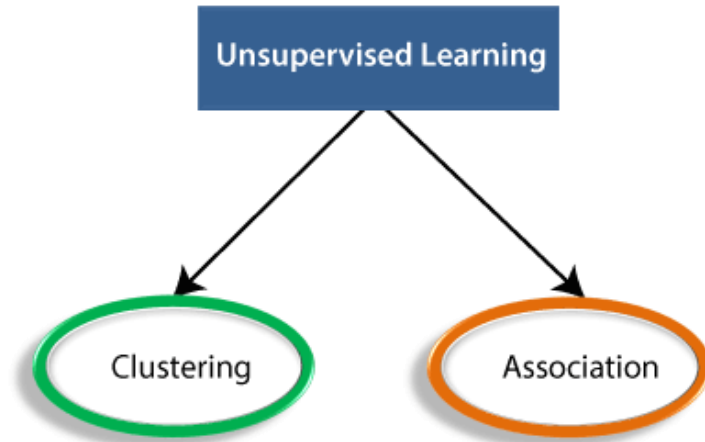
- Here, we have taken an unlabelled input data, which means it is not categorized and corresponding outputs are also not given.
- Now, this unlabelled input data is fed to the machine learning model in order to train it.
- Firstly, it will interpret the raw data to find the hidden patterns from the data and then will apply suitable algorithm to categorize it.
- Once it applies the suitable algorithm, the algorithm divides the data objects into groups according to the similarities and difference between the objects.

Why we use Unsupervised Learning?

- **Unsupervised Learning** is helpful for finding useful insights from the data.
- It is much similar as a human learns to think by their own experiences, which makes it closer to the real AI.
- It works on unlabelled and uncategorized data which makes it more important.
- In real-world, we do not always have input data with the corresponding output so to solve such cases, we need **Unsupervised Learning**.

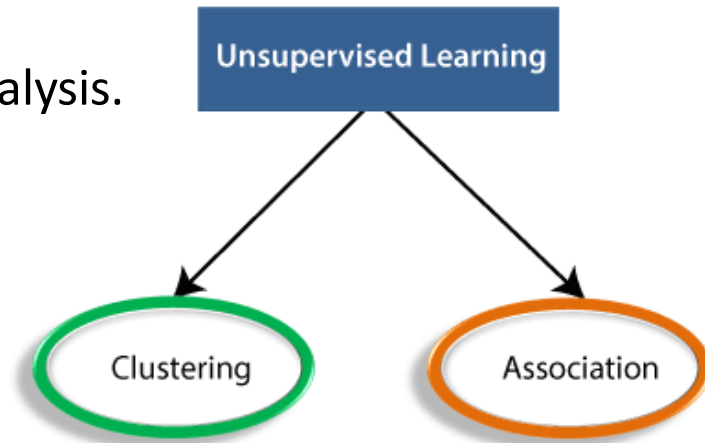
Types of Unsupervised Learning Algorithms:

- **Clustering:** Clustering is a method of grouping the objects into clusters such that objects with most similarities remain into a group and have less or no similarities with the objects of another group.
- Cluster analysis finds the commonalities between the data objects and categorizes them as per the presence and absence of those commonalities.



Types of Unsupervised Learning Algorithms:

- **Association:** Association Rule is used for finding the relationships between variables in the large dataset.
- It determines the set of items that occurs together in the dataset.
- Association Rule makes marketing strategy more effective. Such as people who buy X item (suppose a bread) are also tend to buy Y item (Butter/Jam).
- Example: Market Basket Analysis.



2. Clustering in Machine Learning

CLUSTERING

Clustering is an unsupervised learning algorithm where the data points are segregated based on their characteristics to several clusters (groups) where points lying in the same cluster are more likely to be close to each other in the feature space than points lying in different clusters.

Types of clustering:

- **Hard clustering** is a clustering where each data point is decided to belong to one and only one clusters of the n clusters.
- **Soft clustering** is a clustering where a set of probabilities is assigned to each data point representing the likelihood of belonging this data point to each of the n clusters.

CLUSTERING

Types of clustering algorithms:

- Centroid-based is the one we mostly deal with where each data point is assigned to the cluster where it has the closest distance to its centroid. However such algorithms needs predefinition of the number of clusters which means that it needs prior information, before applying the algorithm, about the clusters that might exist in such data. This algorithm runs iteratively to find local optima and it is fast and efficient (K-means clustering)
- Hierarchical-based is a cluster ranking system where we start either from a single cluster and start getting subclusters or starting from separate clusters and start combining them where number of clusters can then be determined using statistical methods. No prior information is needed about number of clusters.
- Density-based is concerned with areas of high concentration of data points surrounded by areas of low concentration. The places with dense data points are clustered and while with low data points are considered outliers. (DBSCAN)

CLUSTERING

When to use clustering (Applications of clustering)?

Clustering is mainly used with unlabelled data, however we could use other unsupervised learning in such case as association analysis or reinforcement learning or even artificial neural networks.

Clustering can be used when you don't know what features to use as it discovers patterns in a good way that segregates the data based upon it. Hence clustering can be used in some applications as:

- Exploratory data analysis (EDA) which is a crucial step before working with predictive modeling (ML algorithms) where clustering can give a better sense of the data and can let you discard some data points that belongs to useless clusters
- Outlier detection where clustering can give intuition of low density regions of data
- Customer segmentation where we cluster our customers to different clusters to define common traits and behaviours that can lead to better sales and marketing
- Search result grouping where outcomes of search are deduced based on their closeness to each other which is defined by clustering

CLUSTERING

Can clustering be used to improve supervised machine learning algorithms?

YES, we can use unsupervised techniques as clustering to enhance our supervised models by clustering data points of datasets to separate clusters and use the cluster labels as independent variables in the supervised machine learning algorithm.

When you shouldn't use clustering?

If you both don't know any prior information about the number of clusters you want to segment your data into and the dataset is very large as well that can lead to a very high computational power in addition to much time in recognizing clusters. In this case, we may need to switch to artificial neural networks to handle such vague large datasets.

Do we need feature scaling in clustering?

Yes, Feature scaling is a must in clustering as clustering is very sensitive to it. Different feature scaling types can lead to different clustering result.

Example:

- Let's understand the clustering technique with the real-world example of Mall.
- When we visit any shopping mall, we can observe that the things with similar usage are grouped together.



Example:

- Such as the t-shirts are grouped in one section, and trousers are at other sections, similarly, at vegetable sections, apples, bananas, Mangoes, etc., are grouped in separate sections, so that we can easily find out the things.
- Other examples of clustering are grouping documents according to the topic.



Most common uses of Clustering:

- Market Segmentation
- Statistical Data Analysis
- Social Network Analysis
- Image Segmentation
- Anomaly Detection
- Apart from these general usages, it is used by **Amazon** in its recommendation system to provide the recommendations as per the past search of products.
- **Netflix** also uses this technique to recommend the movies and web-series to its users as per the watch history.



amazon

NETFLIX

3. K-Means Algorithm

K-MEANS CLUSTERING

K-means Clustering

K-means clustering is a centroid-based clustering where it works in iterative manner to find the closest centroid to each data point to assign it to while keeping iterating until reaching the best centroids representing the data.

Imagine you're opening a small book store. You have a stack of different books, and 3 bookshelves. Your goal is place similar books in one shelf. What you would do, is pick up 3 books, one for each shelf in order to set a theme for every shelf. These books will now dictate which of the remaining books will go in which shelf. Every time you pick a new book up from the stack, you would compare it with those first 3 books, and place this new book on the shelf that has similar books. You would repeat this process until all the books have been placed. Once you're done, you might notice that changing the number of bookshelves, and picking up different initial books for those shelves (changing the theme for each shelf) would increase how well you've grouped the books. So, you repeat the process iteratively in hopes of a better outcome. K-Means algorithm works something just like this.

K-MEANS CLUSTERING

K-means Clustering algorithm steps:

Choose the number of clusters K :

This number is based on the prior knowledge of the data and the definition of your problem.

Initialize Cluster Centroids :

Select K points at random from the data points to let them be the centroids (centers).

⇒ Iterate over the following two steps:

Assignment step:

Assign each data points to the closest center of the k centers which is done by computing distance (Euclidean, Manhattan, ..) to all centers and choose the least one.

Update step:

For all data points related to same cluster, compute the new cluster center which is done by calculating the mean of each cluster data points.

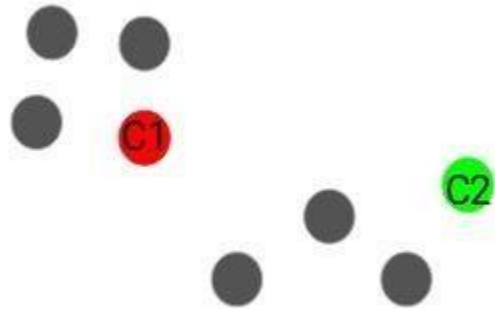
K-MEANS CLUSTERING

After each iteration, the centroids will move slowly so that the total distance from each point to the closest centroid gets lower, and this keeps performed until convergence where no changes takes place to the data points assignment hence the centroids won't change which guarantees reaching a local optimum cluster assignment.

Example:

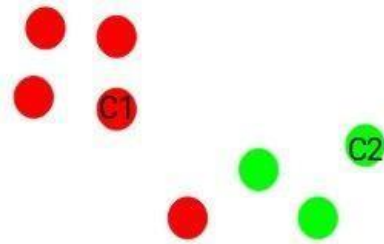
Step1: We choose $K=2$.

Step2: We take K random points from the data points to be the centroids as shown:

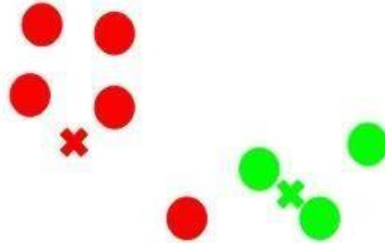


K-MEANS CLUSTERING

Step3: Assign all data points to the closest cluster centroid as shown below:

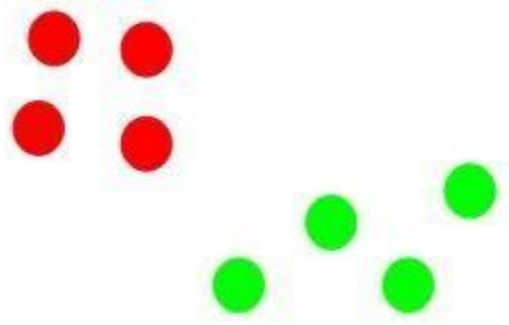


Step4: Recompute the cluster centroids once again:



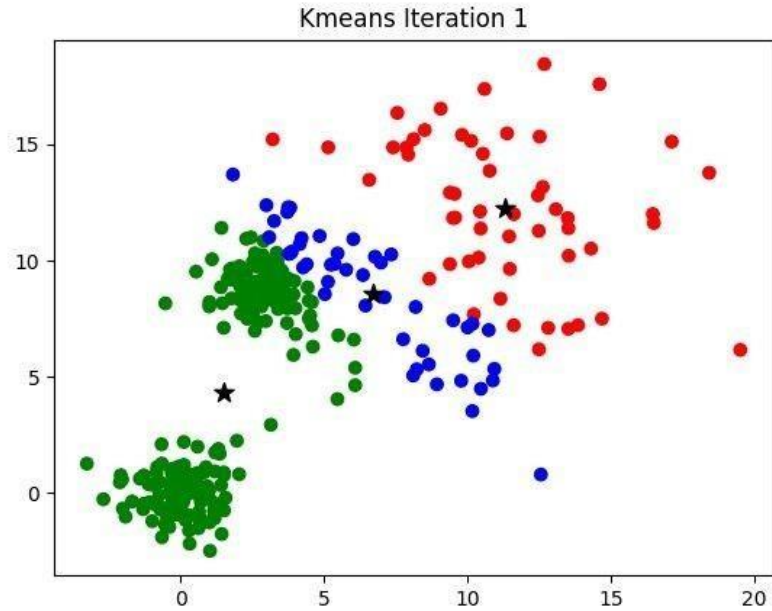
K-MEANS CLUSTERING

Repeat step 3 and step 4:



The step of computing the centroid and assigning all the points to the cluster based on their distance from the centroid is a single iteration. But wait - when should we stop this process? It can't run till eternity, right?

K-MEANS CLUSTERING



K-MEANS CLUSTERING

What is the stopping criteria of the iterations done by K-means clustering?

Since step 3 and 4 are performed iteratively, we should have stopping criteria for these iterations noting that setting this stopping criteria doesn't return necessarily the best clusters.

- Data points remains assigned to the same cluster. (time consuming)
- Centroids remains the same with no change. (More time consuming)
- The distance from each data point to its centroid is lower than a certain specified threshold that has been set.
- Maximum number of iterations has been set noting that low number of iterations can lead to poor results while high number of iterations can be time consuming.

You can choose a combination of stopping criteria as choosing whether data points remains assigned to same cluster or maximum number of iterations reached. Whoever comes first.

K-MEANS CLUSTERING

How is the new cluster centroid (update step) is computed?

As we mentioned before that we get the new cluster centroid by getting the mean of all data points belonging to the same cluster. This step is done feature wise, where the new centroid feature values is the mean of each feature set of these data points as shown below:

Updating Cluster Centroids

old centroid of C#1:

2	3	1
---	---	---

f1	f2	f3
4	2	0
3	3	1
5	1	3
4	0	2

} datapoints in C#1

New centroid = Avg of data points feature wise

$$\frac{4+3+5+4}{4}=4, \frac{2+3+1+0}{4}=1.5, \frac{0+1+3+2}{4}=1.5$$

new centroid of C#1

4	1.5	1.5
---	-----	-----

K-MEANS CLUSTERING

What if we don't have a prior knowledge of number of clusters? Are there any way of estimating the best number of clusters?

Before answering this question, let us define a terminology called inertia. Inertia is the sum of distances of all points in a cluster to the centroid of this cluster which is summed up over all the clusters. From this definition, we can say that as inertia gets lower, we are having a better clustering algorithm.

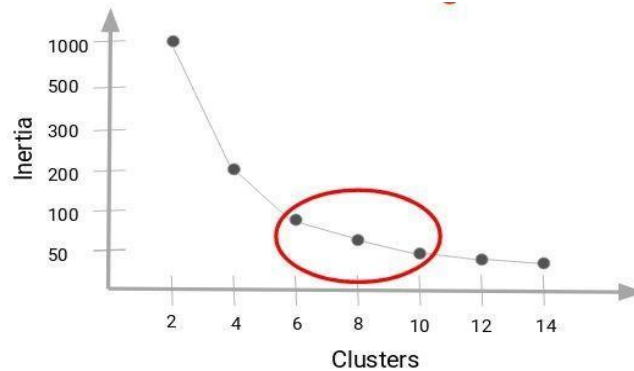
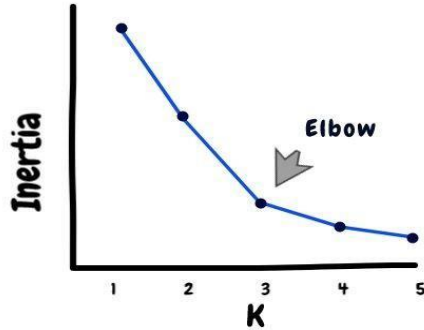
The maximum possible number of clusters will be equivalent to the number of data points in the dataset where each data point is considered as cluster on its own. This would give us the lowest inertia possible as all distances are zeros so the summation will be 0. Similarly we can say that making all the data as one cluster will result in the highest inertia possible. Hence we should find a way to choose the optimal reasonable value of K based on the inertia, This raises the Elbow method.

Note: The inertia ranges from 0 to infinity value.

K-MEANS CLUSTERING

What if we don't have a prior knowledge of number of clusters? Are there any way of estimating the best number of clusters?

We will apply a methodology similar to hyper-parameter tuning by trying out several K values from 1 to n and for each value we calculate the inertia corresponding to this value. The optimal value is one of the values at the elbow of the curve.



Note: We can use any of the evaluation metrics that will be explained later in normal hyper-parameter tuning for K by choosing the value of K having highest score of metric.

Choosing the Number of Clusters:

- The performance of the K-Means clustering algorithm depends upon highly efficient clusters that it forms.
- But choosing the optimal number of clusters is a big task.
- There are some different ways to find the optimal number of clusters, but here we are discussing the most appropriate method to find the number of clusters or value of K, **Elbow Method**.

Choosing the Number of Clusters “Elbow Method”:

- The **Elbow Method** is one of the most popular ways to find the optimal number of clusters.
- This method uses the concept of **WCSS** value.
- WCSS stands for **Within Cluster Sum of Squares**, which defines the total variations within a cluster.
- The formula to calculate the value of WCSS (for 3 clusters):

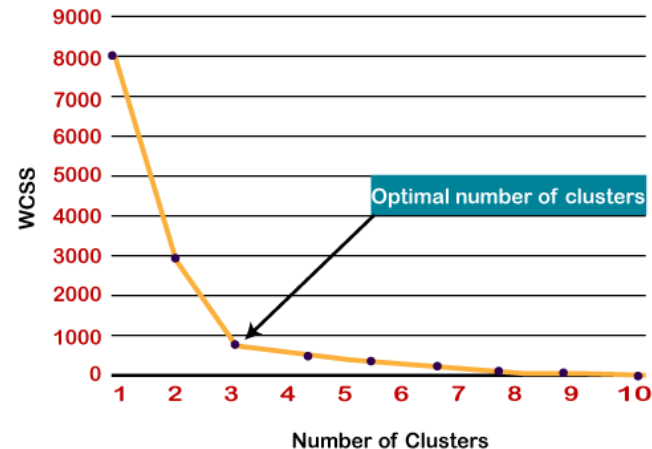
$$\text{WCSS} = \sum_{P_i \text{ in Cluster1}} \text{distance}(P_i C_1)^2 + \sum_{P_i \text{ in Cluster2}} \text{distance}(P_i C_2)^2 + \sum_{P_i \text{ in Cluster3}} \text{distance}(P_i C_3)^2$$

$\sum_{P_i \text{ in Cluster1}} \text{distance}(P_i C_1)^2$: It is the sum of the square of the distances between each data point and its centroid within a cluster 1.
And the same for the other two terms.

Choosing the Number of Clusters “Elbow Method”:

The **Elbow Method** follows the below steps:

- It executes the K-means clustering on a given dataset for different K values (ranges from 1-10).
- For each value of K, calculates the WCSS value.
- Plots a curve between calculated WCSS values and the number of clusters K.
- The sharp point of bend or a point of the plot looks like an arm, then that point is considered as the best value of K.

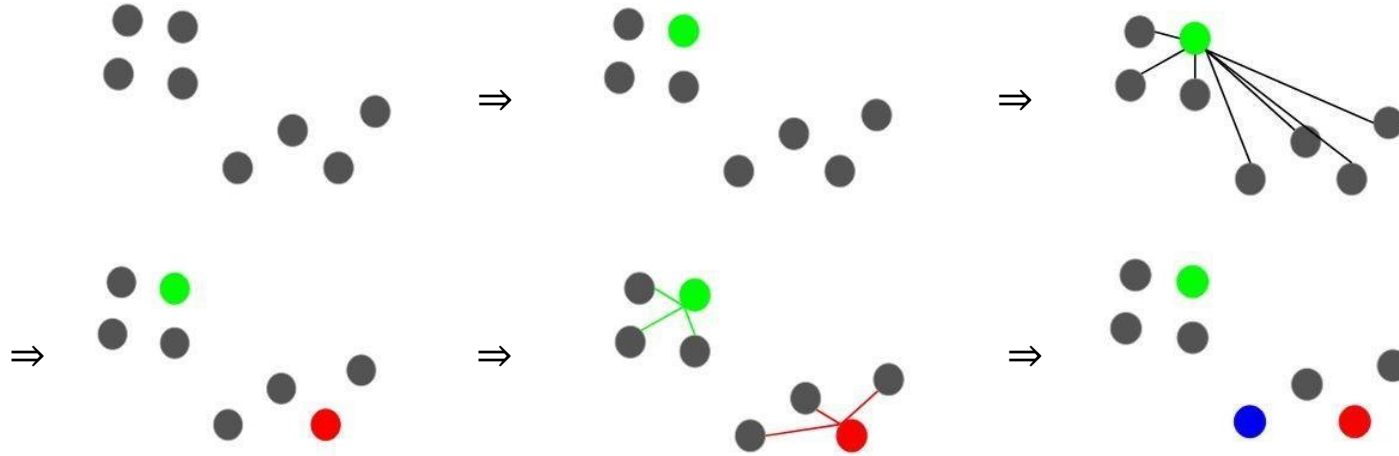


Initialization of the Centroids “K-Means++”:

- To overcome the previously-mentioned drawback, we use **K-Means++**.
- **K-Means++** ensures a smarter initialization of the centroids and improves the quality of the clustering.
- Randomly select the first centroid from the data points.
- For each data point compute its distance from the nearest, previously chosen centroid.
- Select the next centroid from the data points such that the point having maximum distance from the nearest centroid is most likely to be selected next as a centroid.
- Repeat steps 2 and 3 until k centroids have been sampled.

K-MEANS CLUSTERING

K-means++ initialization Example:

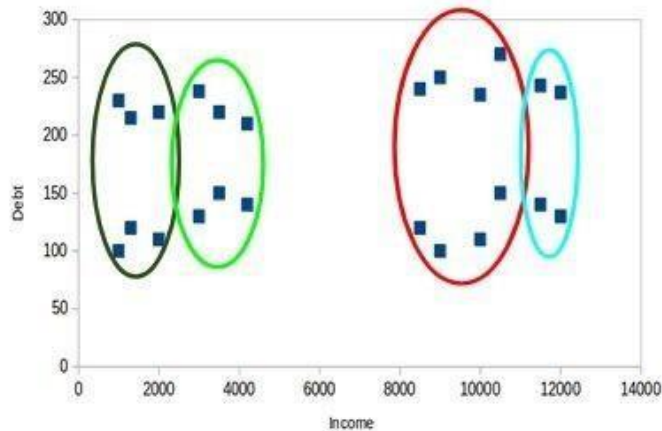


Note: As observed, K-means++ is more computationally expensive than normal random initialization however it converges more rapidly and tends to give better quality of local optimum and overall lower runtime.

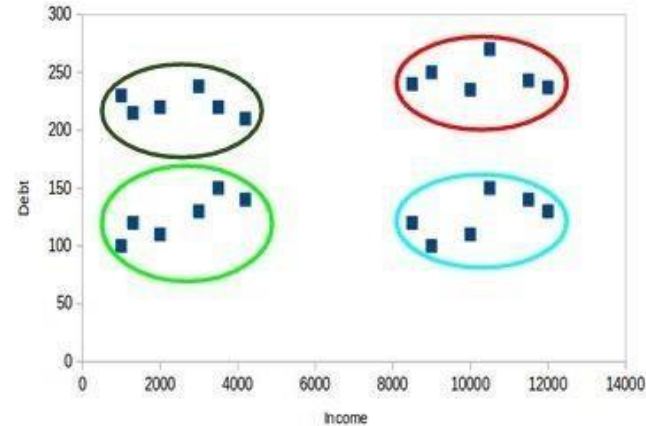
K-MEANS CLUSTERING

Which case is better Case I or Case II ?

Assuming we have 2 features Income (X-axis) and Debt (Y-axis) distributed in the features space as shown in the following figure:

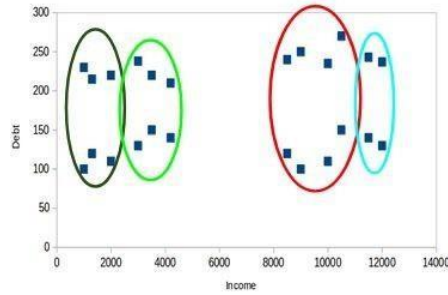


Case - I

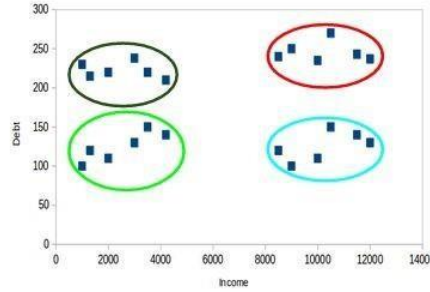


Case - II

K-MEANS CLUSTERING



Case - I



Case - II

Noticing the case I, the red and blue clusters are quite similar to each other as for example the 4 top points in the red cluster and the two top points in the blue cluster are sharing the same characteristics by having both high values in income and debt while in case II each cluster is having different characteristics from combination of low and high income and debt. For example red cluster has high income value with high debt value, while blue cluster has high income value with low debt value. However this can be deduced easily using visualization, but with higher dimensions (higher number of features, we wouldn't be able to do so. Hence we should define generic evaluation metrics.

K-MEANS CLUSTERING

Advantages of K-Means Clustering

- Simple to understand and implement.
- Guarantees convergence.
- It is much faster and computationally efficient in a comparison with other clustering technique as hierarchical clustering.
- More suitable with large datasets than other clustering algorithms.

K-MEANS CLUSTERING

Disadvantages of K-Means Clustering

- Needs pre-definition of number of clusters which needs prior info about the data.
- The final result is dependable on the initial values of cluster centroids.
- Time consuming if initialization was bad.
- Not robust to outliers (use median instead of mean in computing cluster centroids however this will increase the time due to sorting data each time).
- As number of features increases, the distance metric would become less accurate due to curse of dimensionality explained before.
- Bad at clustering datasets with varying sizes and density so it needs to be generalized.
(This problem will be explained in details in GMM)
- Handles numerical features only which can be impractical in many cases.
- Sensitive to scales so standardization or normalization would lead to different final result.

HIERARCHICAL CLUSTERING

Hierarchical Clustering (HCA)

Hierarchical Clustering is a type of clustering where we deal with database in hierarchical manner to build a tree-like shape representing the clusters of data points. Hierarchical clustering has mainly two types:

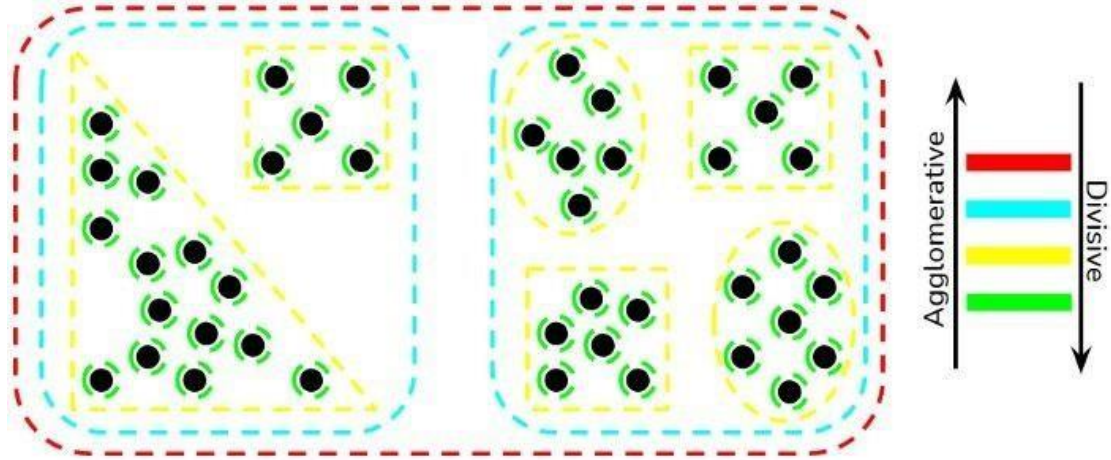
Agglomerative clustering (most common):

It is a bottom-up approach where we start as if each point is a cluster on its own, then we start linking the closest two clusters together with each other, and iterate in this manner until combining all data points in one cluster.

Divisive clustering:

It is a top-down approach where we start as single cluster having all points inside and start splitting out the farthest point from the center of the clusters each time until reaching that all data points becomes clusters on their own.

HIERARCHICAL CLUSTERING



HIERARCHICAL CLUSTERING

Agglomerative Hierarchical Clustering Example

Assume having the following dataset representing the marks of students:

Student_ID	Marks
1	10
2	7
3	28
4	20
5	35



HIERARCHICAL CLUSTERING

Agglomerative Hierarchical Clustering Example

Now we compute what we call a proximity (distance) matrix where it gives us all the available combinations of distances (any distance measurement as Euclidean, Manhattan, Minkowski, ..) between all data points as shown in the following matrix:

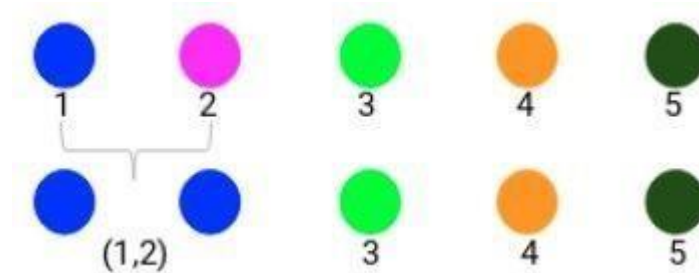
ID	1	2	3	4	5
1	0	3	18	10	25
2	3	0	21	13	28
3	18	21	0	8	7
4	10	13	8	0	15
5	25	28	7	15	0

HIERARCHICAL CLUSTERING

Agglomerative Hierarchical Clustering Example

As we have seen from the previous cost matrix, the closest two data points are 1 and 2 with a cost of 3.

ID	1	2	3	4	5
1	0	3	18	10	25
2	3	0	21	13	28
3	18	21	0	8	7
4	10	13	8	0	15
5	25	28	7	15	0



HIERARCHICAL CLUSTERING

Agglomerative Hierarchical Clustering Example

Our new dataset now will be $\Rightarrow$

Student_ID	Marks
(1,2)	10
3	28
4	20
5	35

Note: We took the maximum of the two number in the cluster, however we can take the minimum or the average. We will take about this later in the linkage methods part.

HIERARCHICAL CLUSTERING

Agglomerative Hierarchical Clustering Example Our new distance matrix now will be as follows:

ID	(1,2)	3	4	5
(1,2)	0	18	10	25
3	18	0	8	7
4	10	8	0	15
5	25	7	15	0

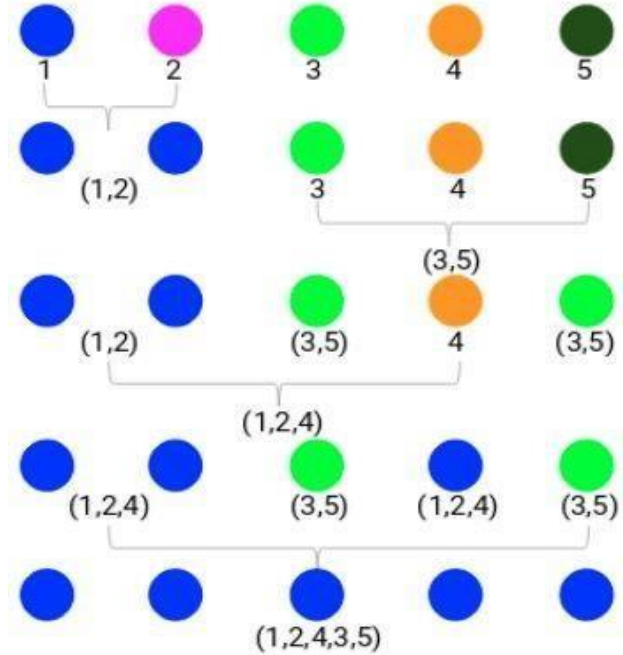
Which means that the next two closest clusters are 3 and 5 with a cost of 7.

HIERARCHICAL CLUSTERING

Agglomerative Hierarchical Clustering Example We keep iterating like this until we have all data points as one cluster as shown:

But here comes the question? What should we do next? How can we decide which number of Clusters is best suited for this data set?

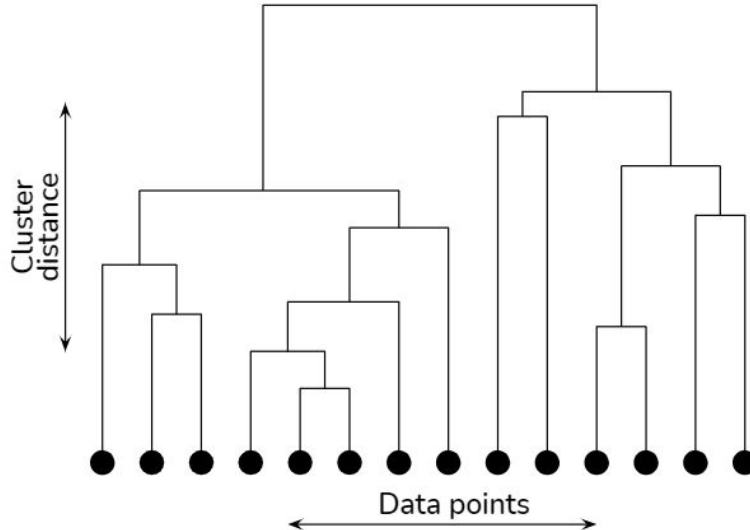
Here comes the idea of dendrogram.



HIERARCHICAL CLUSTERING

Dendrogram

The whole concept of hierarchical clustering lies in the construction and analysis of the dendrogram which is a tree-like structure that explains the relationship between the data samples and each other based on their distances between their clusters.

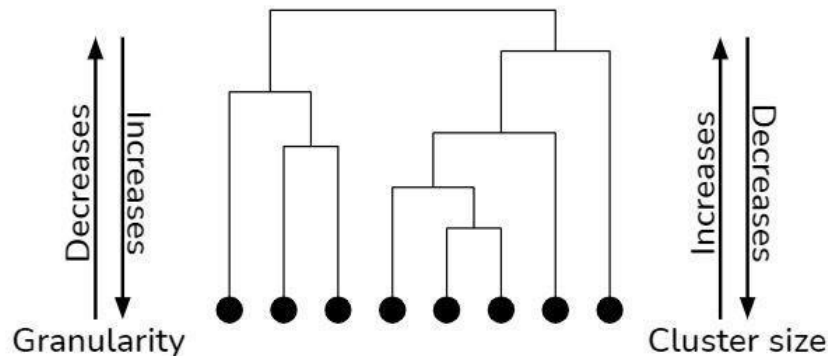


HIERARCHICAL CLUSTERING

Dendrogram

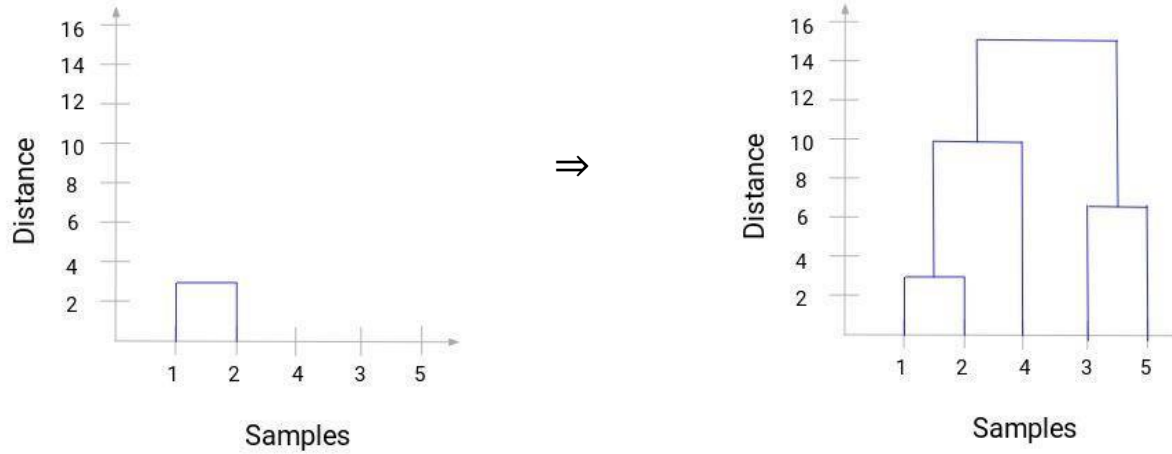
Starting from the top, we have all data points as a single cluster and keeps breaking the cluster into more smaller clusters (smaller cluster size) until reaching the bottom where we have all data samples represented as m clusters where m is the number of data samples.

Note: Granularity means level of detail.



HIERARCHICAL CLUSTERING

Dendrogram for our Example

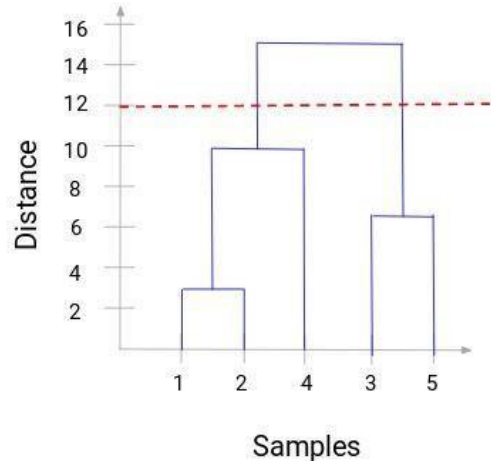


More the distance of the vertical lines in the dendrogram, more the distance between those clusters.

HIERARCHICAL CLUSTERING

Dendrogram for our Example

Now how would we decide the number of clusters? We try to cut our dendrogram in a way that it cuts the tallest vertical line for each number of clusters. This step is done visually by using dendrogram.

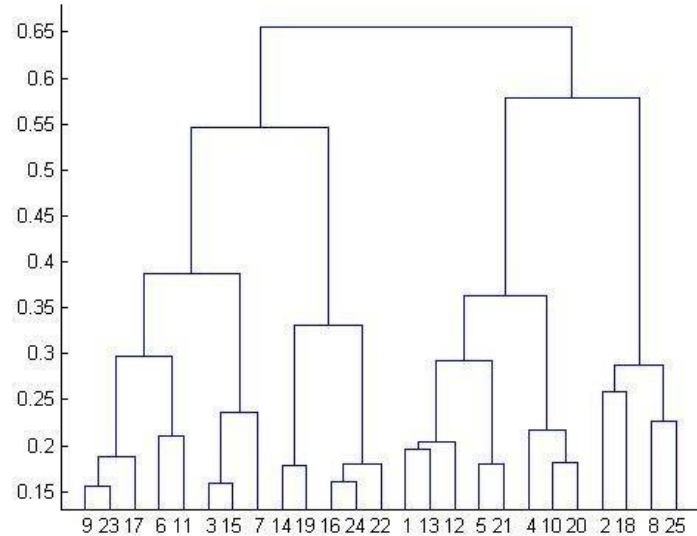


The number of clusters will be the number of vertical lines which are being intersected by the line drawn using the threshold. Hence 2 is the optimum number of clusters.

HIERARCHICAL CLUSTERING

Dendrogram

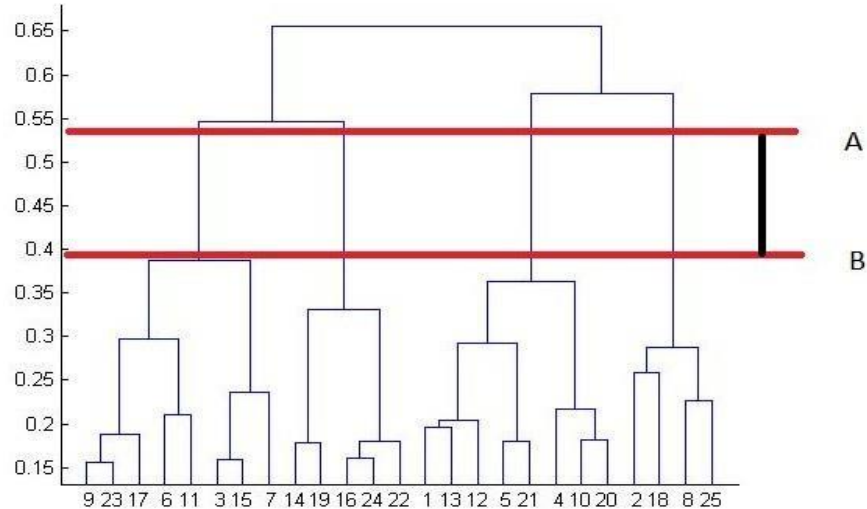
What would be the number of clusters if the dendrogram is as follows?



HIERARCHICAL CLUSTERING

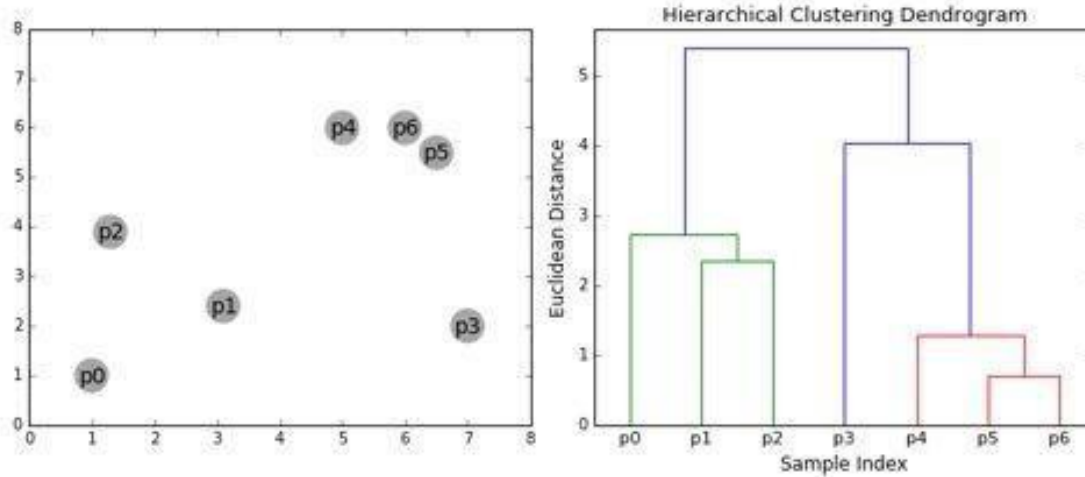
Dendrogram

The best line is the horizontal line that cuts the tallest vertical line in a way that maximizes vertical cut distance along all number of clusters candidates. Hence you can cut in any place between A and B resulting in 4 clusters.



HIERARCHICAL CLUSTERING

Agglomerative Hierarchical Clustering with Dendrogram



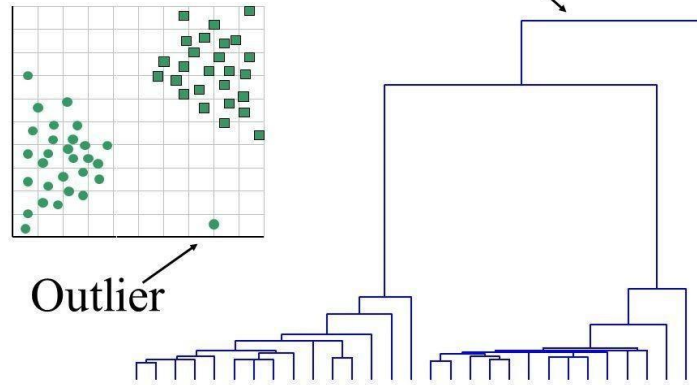
HIERARCHICAL CLUSTERING

Can Hierarchical clustering handles outliers? Sensitive or not to outliers.

Hierarchical clustering is generally sensitive to outliers where it can give a decision of cutting a dendrogram at a certain height which can be misleading.

One potential use of a dendrogram is to detect outliers

The single isolated branch is suggestive of a data point that is very different to all others

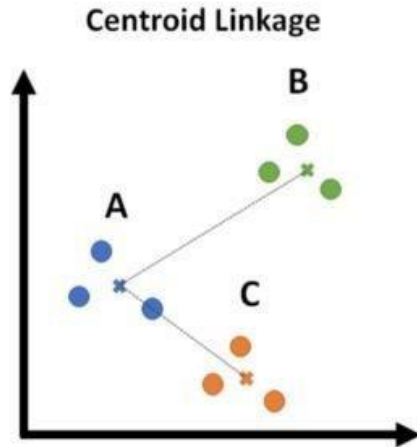


HIERARCHICAL CLUSTERING

Linkage methods

Linkage method defines the way of how we link two clusters together. We have talked about linkage method inherently while dealing with the example taken.

- 1) **Centroid linkage** is the distance between the centroids of 2 clusters before merging.



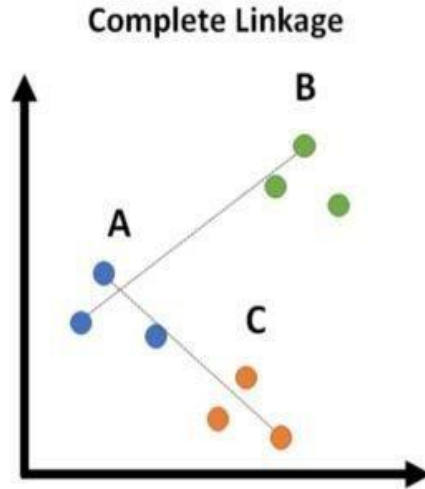
Link cluster A with cluster C because:

Distance from **centroid** of cluster A to centroid of cluster C is **shorter** than the distance from **centroid** of cluster A to centroid of cluster B

HIERARCHICAL CLUSTERING

Linkage methods

2) Complete linkage is the longest distance among any two points in the 2 clusters. Use it if there is noise between clusters. This method reveals outliers better visually.



Link cluster A with cluster C because:

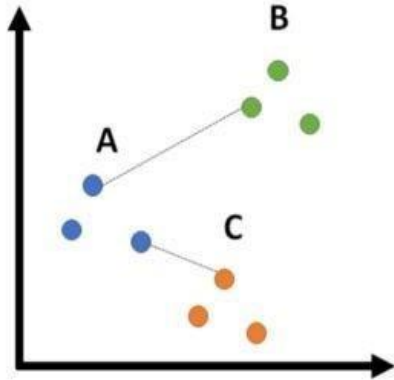
maximum distance from
cluster A to cluster C
Is **shorter** than the
maximum distance from
cluster A to cluster B.

HIERARCHICAL CLUSTERING

Linkage methods

3) Single linkage is the shortest distance among any two points in the 2 clusters. This can(not always) be good if outliers exist in the data.

Single Linkage



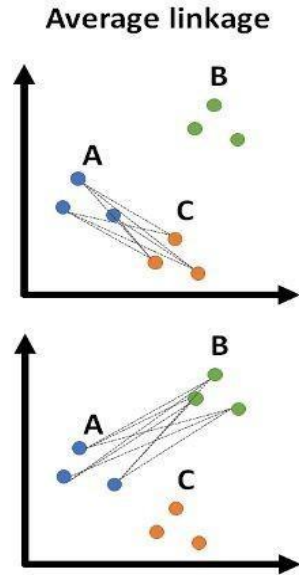
Link cluster A with cluster C because:

minimum distance from
cluster A to cluster C
Is **shorter** than the
minimum distance from
cluster A to cluster B.

HIERARCHICAL CLUSTERING

Linkage methods

4) Average linkage is the average distance between each point in a cluster to each point the other cluster.



Link cluster A with cluster C because:

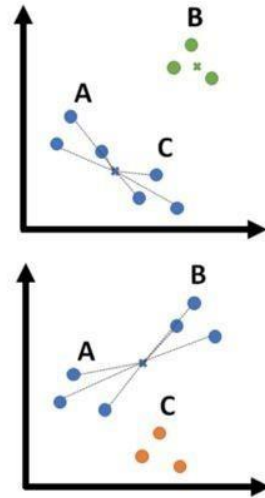
mean distance from
cluster A to of cluster C
Is **shorter** than the
mean distance from
cluster A to cluster B.

HIERARCHICAL CLUSTERING

Linkage methods

5) Ward linkage is the sum of squares of deviation from the mean of the 2 clusters (variance) after fusing them. It handles well the noise between clusters as well.

Ward's linkage method



Link cluster A with cluster C because:

The total **variance** of clusters A and C is **lower** than the total **variance** of clusters A and B.

HIERARCHICAL CLUSTERING

Advantages of Hierarchical Clustering:

- Don't have to manually select K prior applying the algorithm.
- Easy to implement.
- Dendrogram is very informative and visually appealing.
- No need to initialize random centroids.

Disadvantages of Hierarchical Clustering:

- Very large datasets can produce difficult dendrogram to be separated.
- Computationally expensive and time consuming more than K-means.
- Sensitivity to outliers.
- Doesn't work well with missing data.
- Has problems with highly dimensional data because of poor distance measurement quality in the hyper-space.